

The logo for Oracle Academy is centered on a light gray background. It features the word "ORACLE" in a bold, orange, sans-serif font. Below it, the word "Academy" is written in a smaller, dark gray, sans-serif font. The entire logo is framed by a thin black border, with dark gray horizontal bars at the top and bottom.

ORACLE

Academy

Database Foundations

3-1

More with Relationships

ORACLE
Academy



Copyright © 2020, Oracle and/or its affiliates. All rights reserved.

Objectives

- This lesson covers the following objectives:
 - Identifying barred relationships
 - Resolve M:M relationships
 - Identify and illustrate nontransferable relationships
 - Identify and draw supertype and subtype entities
 - Identify hierarchical, recursive, and arc relationships



Identifying (Barred) Relationships

- What is the UID of the ACCOUNT entity?



- The UID requires both the ACCOUNT Number and the relationship between BANK and ACCOUNT



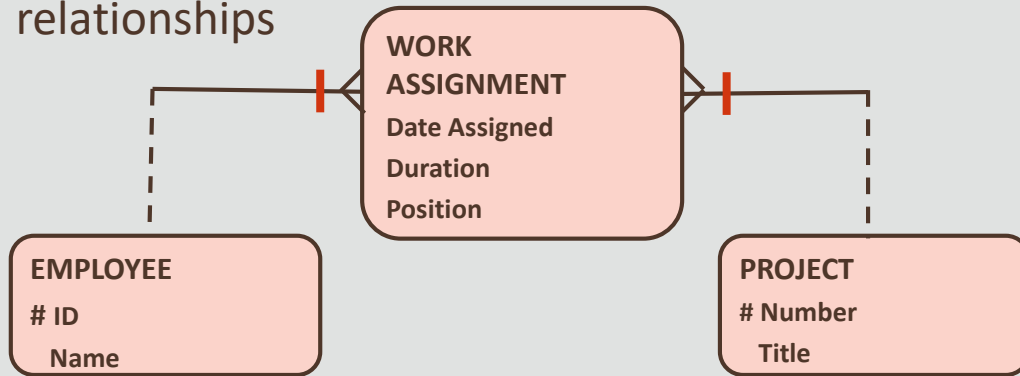
- The UID for the ACCOUNT entity is made up of ACCOUNT Number as well as BANK Number – this is represented by the bar on the relationship line

An identifying relationship is created when the UID for an entity includes the relationship with another entity.

Note that a relationship included in a UID must be mandatory and one-and-only-one in the direction that participates in the UID.

Identifying Relationships with Multiple Entities

- An entity can be uniquely identified through multiple relationships



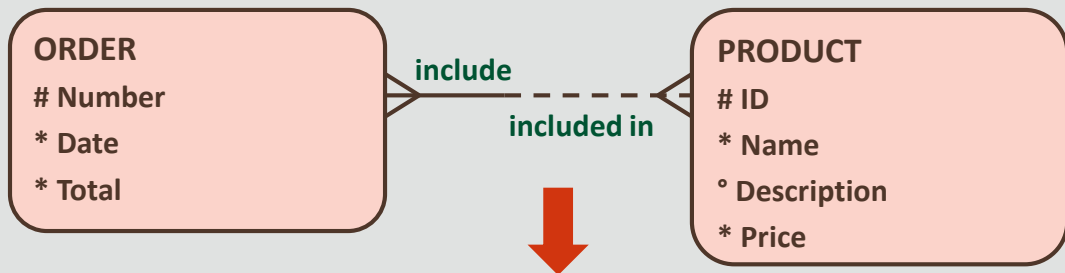
- The UID for WORK ASSIGNMENT is EMPLOYEE ID and PROJECT Number (represented using bars)

In the slide example, an EMPLOYEE and a PROJECT are needed to make WORK ASSIGNMENT unique, so both relationships are included in the UID for WORK ASSIGNMENT.

Note that WORK ASSIGNMENT is an intersection entity that is the resolution between an M:M relationship. This topic is discussed in a later lesson.

M:M Relationships

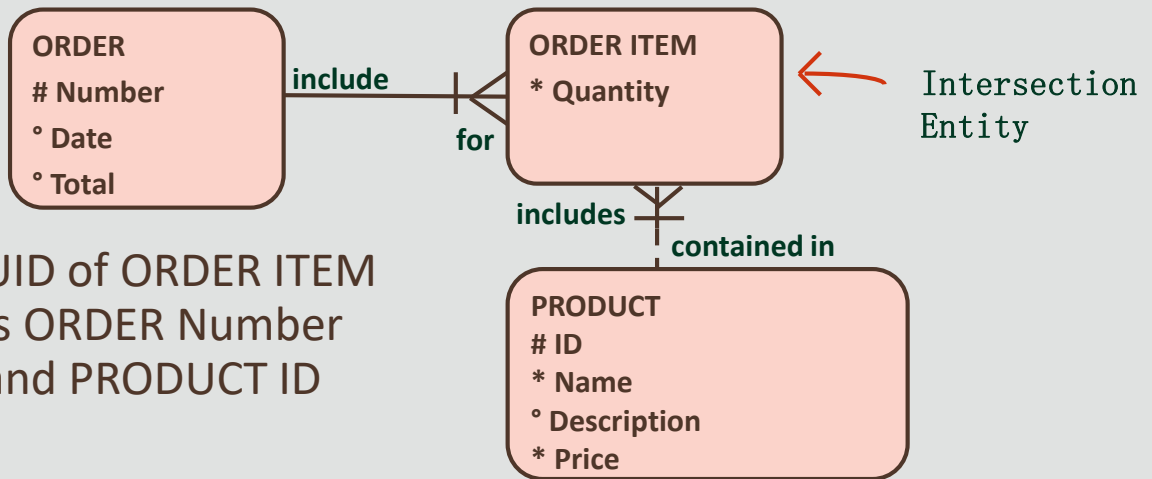
- Attributes describe only entities
- If attributes describe a relationship, the relationship must be resolved



Where would you add the Quantity attribute?

Resolving M:M Relationships: Example 1

- Resolving a M:M relationship with a new intersection entity and two 1:M barred relationships



- UID of ORDER ITEM is ORDER Number and PRODUCT ID

Intersection entity characteristics:

- The relationships from the intersection entity are always mandatory
- Intersection entities usually contain consumables like quantity used and dates. They tend to be high-volume and volatile entities
- An intersection entity is identified by its two originating relationships (identifying relationships)

Note: Choosing a name for the intersection entity is often difficult. Leave it blank at first, move the relationships, add the attributes, and then it will often be clearer what the name should be.

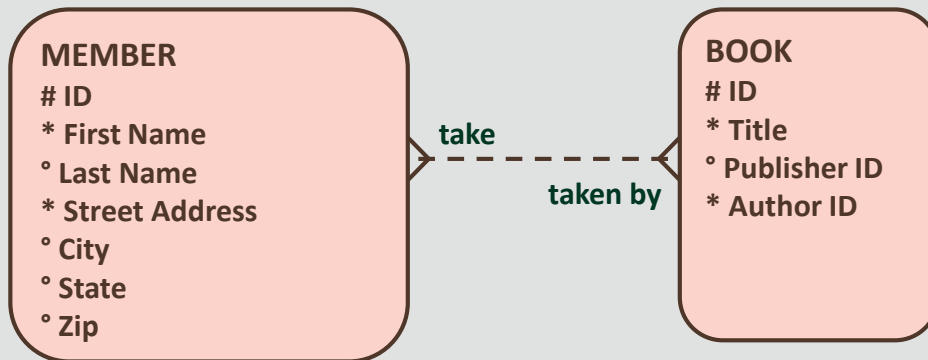
If you do not have any additional attributes in the intersection entity, you can leave it as an M:M relationship and Oracle SQL Developer Data Modeler will create the intersection table in the relational model.

Case Scenario: Resolving M:M Relationships



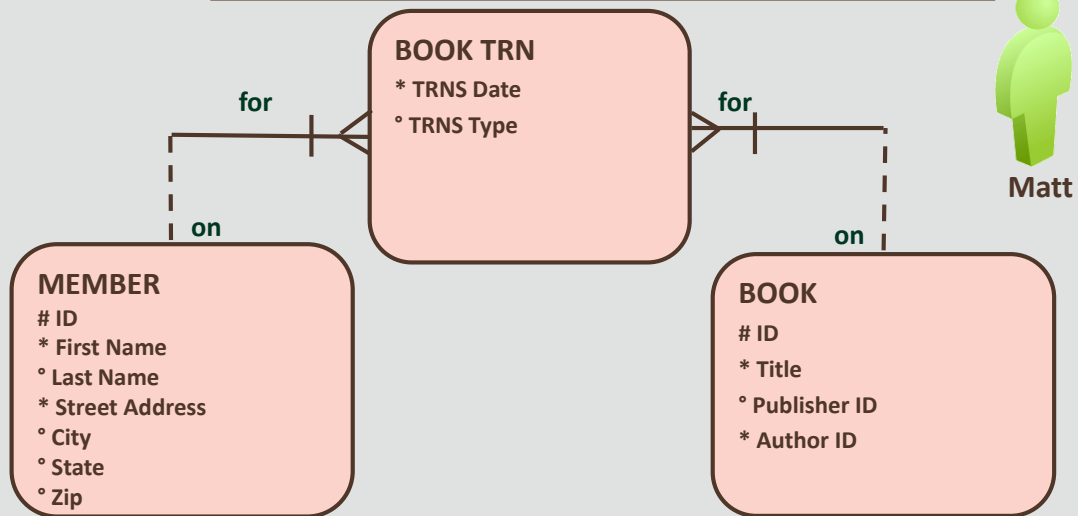
Faculty

Matt, how would you resolve the following M:M relationship between the MEMBER and the BOOK entities?



Case Scenario: Creating an Intersection Entity

I would create an intersection entity with identifying relationships to the originating entities.



ORACLE
Academy

DFo 3-1
More with Relationships

Copyright © 2020, Oracle and/or its affiliates. All rights reserved. 10

A third entity, called an intersection entity, is required to resolve the M:M relationship.

To resolve the M:M relationship from the previous slide, perform the following steps:

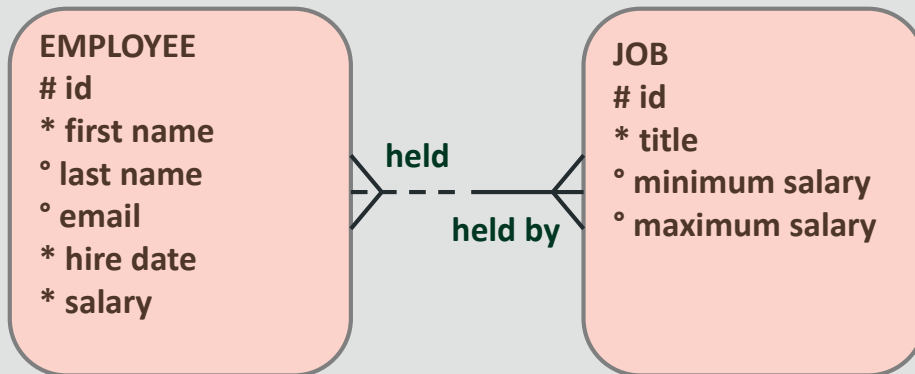
- Delete the M:M relationship.
- Create a new entity called **BOOK TRN**.
- Create two identifying relationships: one from **MEMBER** and the other from **BOOK**.
- Identify and create additional attributes in the **BOOK TRANSACTION** intersection entity.
- Evaluate whether the two identifying relationships constitute a UID for the intersection entity or whether other attributes are needed.

TRNS ID is an artificial UID that could be used instead of barred relationships. It is an alternative method, not better or worse.

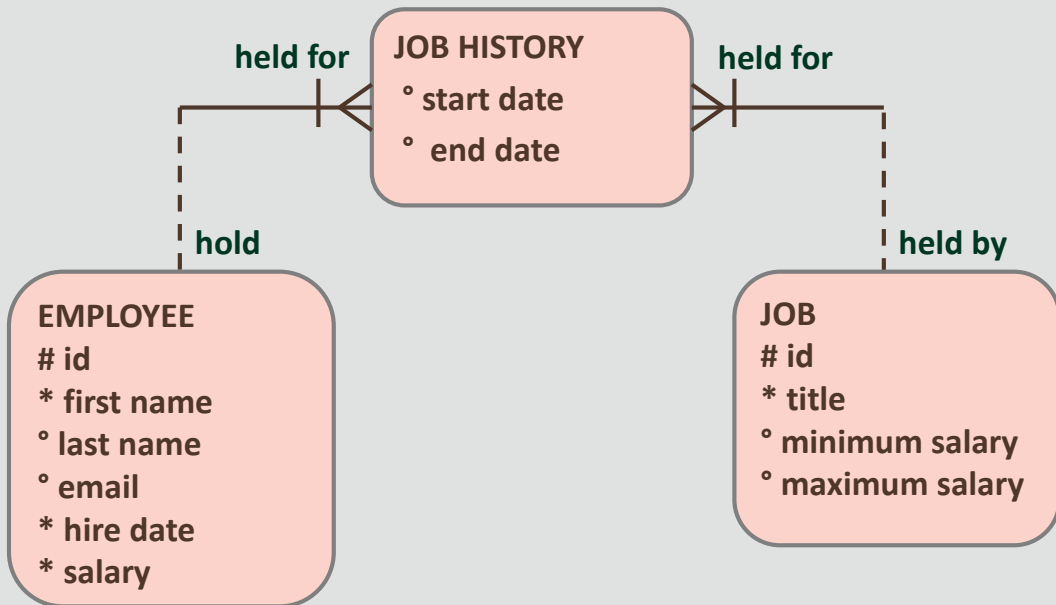
Note: Not all intersection entities will have additional attributes.

Resolving M:M Relationships: Example 2

- The EMPLOYEE and JOB entities do not store the history of an employee's jobs



Resolving M:M Relationships Using Intersection Entity with Barred Relationships



There is a M:M relationship between **EMPLOYEE** and **JOB**. There is no way to store information about an employee's jobs. The **JOB HISTORY** intersection entity stores this information with two identifying relationships for the **EMPLOYEE** and **JOB** entities.

A barred relationship is mapped to a foreign key column on the many side, just like any other M:1 relationship. The UID of the intersection entity often comes from the originating relationships and is represented by bars.

Project Exercise 1

- DFo_3_1_1_Project
 - Oracle Baseball League Store Database
 - Resolving Many to Many Relationships



ORACLE
Academy

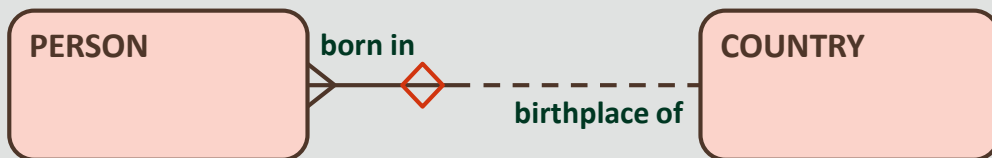
DFo 3-1
More with Relationships

Copyright © 2020, Oracle and/or its affiliates. All rights reserved.

13

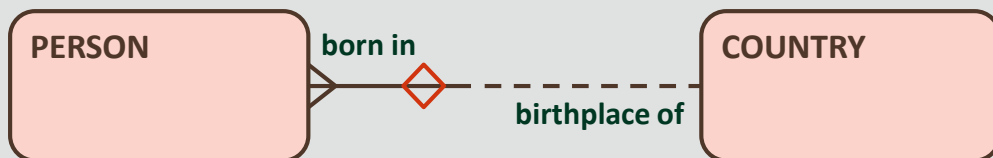
Non-Transferable Relationships

- Transferability is the ability of the relationship between two instances of an entity to change over time
- A non-transferable relationship cannot be moved between instances of the entities it connects
- A non-transferable relationship is represented by a diamond on the relationship



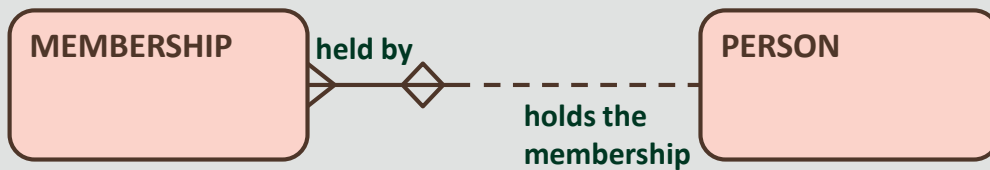
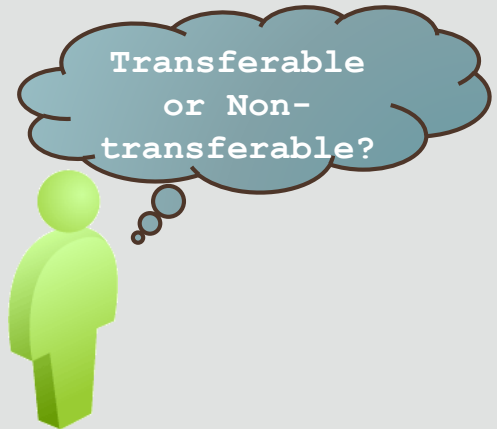
Non-Transferable Relationships

- Non-transferable relationships can only be mandatory
- For example, the birth country of a person is non-transferable



Case Scenario

- A membership must be held by exactly one person
- The membership relationship cannot be moved to another person



Project Exercise 2

- DFo_3_1_2_Project
 - Oracle Baseball League Store Database
 - Identify and Illustrate Non-Transferable Relationships



ORACLE
Academy

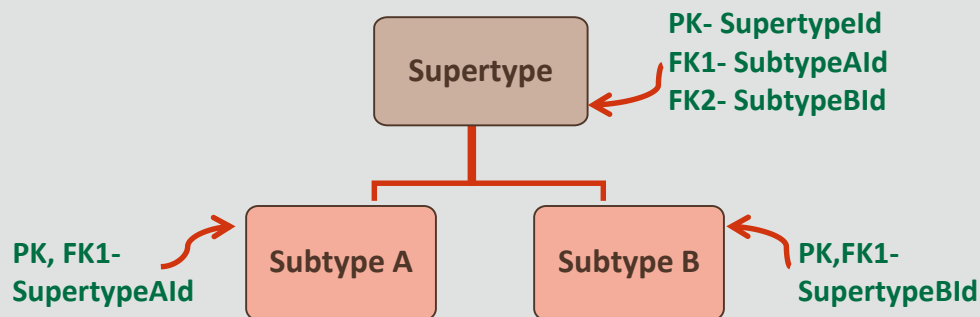
DFo 3-1
More with Relationships

Copyright © 2020, Oracle and/or its affiliates. All rights reserved.

17

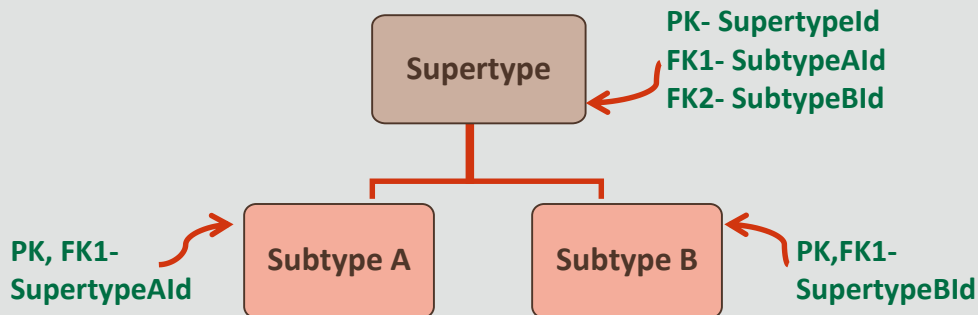
Supertype and Subtype Entities

- Supertype has a parent-child relationship with one or more subtypes



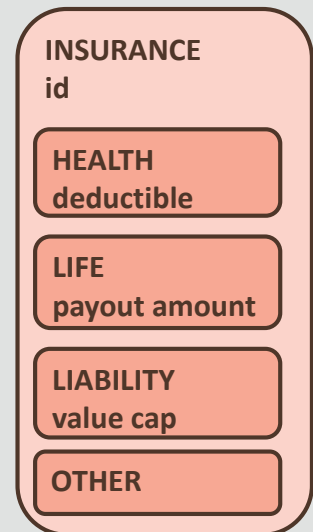
Supertype and Subtype Entities

- Subtype is a subgrouping of the entity in an entity type which has attributes that are distinct from those in other subgroupings



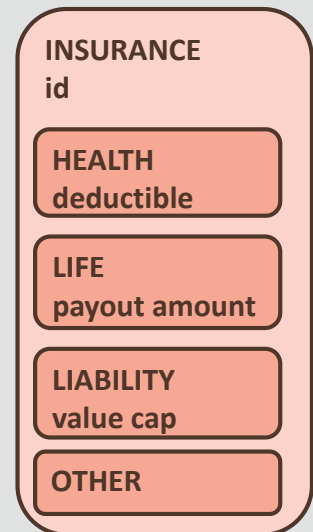
Drawing a Subtype

- Each subtype is a specialization of a supertype and therefore must be enclosed within an entity
- The common attributes and relationships for all subtypes must be listed only in the supertype, but they are inherited in every subtype



Drawing a Subtype

- A subtype can and would generally have attributes and relationships of its own
- There can never be just one subtype; another subtype should be created to contain the rest



Characteristics of a Subtype

- A subtype:
 - Inherits all attributes of the supertype
 - Inherits all relationships of the supertype
 - Usually has its own attributes or relationships
 - Is drawn within the supertype
 - Never exists alone
 - May have subtypes of its own
 - Has identical primary keys of the supertype and subtype

Supertype and Subtype Entity: Example



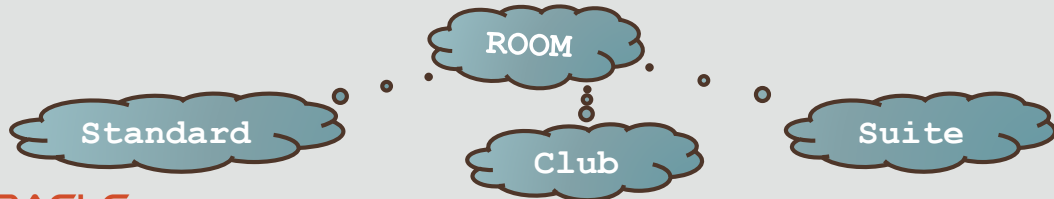
Sharon wants to include types of room categories to be reserved for the guests. For example:

Standard

Club

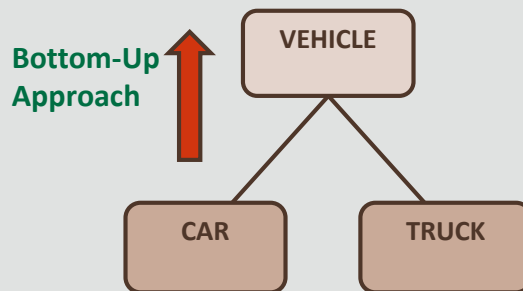
Suite

She already has an entity named ROOM that holds room details. This would be a supertype entity. The specific room categories would inherit the properties of the supertype entity, in addition to their own specific attributes. The specific room category would be the subtype entity.



Generalization and Specialization

- Generalization is a bottom-up approach where two or more lower level entities are combined to form a higher level entity based on the common feature

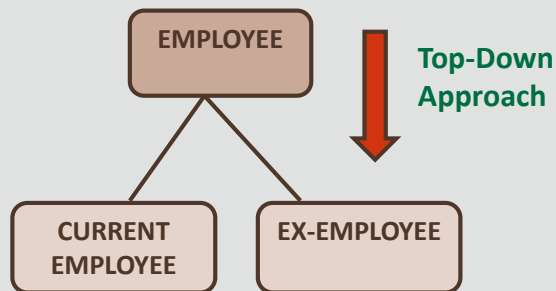


Generalization is the process of extracting shared characteristics from two or more entities, and combining them into a generalized supertype. Shared characteristics can be attributes, associations, or methods.

In the slide example, CAR and TRUCK share the same common attributes, and so their common attributes are grouped to form the supertype entity named Vehicle.

Generalization and Specialization

- Specialization is a top-down approach where the higher level entity is broken down into lower level entities



Specialization means creating new subtypes from an existing entity. If it turns out that certain attributes, associations, or methods apply only to some of the entity's objects, a subtype can be created.

In the slide example, the EMPLOYEE supertype is broken down into two subtypes, CURRENT EMPLOYEE and EX-EMPLOYEE.

Entity Subtype Rules

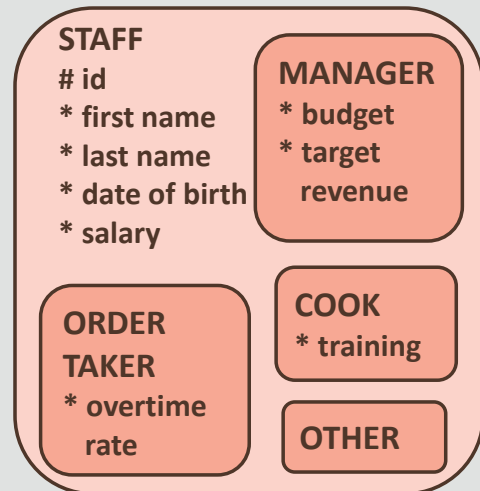
- Exhaustive:
 - Every instance of a supertype is also an instance of one of the subtypes
 - OTHER should be included as a subtype to categorize instances that are not defined by one of the existing subtypes
 - Example: An employee must be full time, part time, or other

Entity Subtype Rules

- Mutually Exclusive:
 - Every instance of the supertype is of one and only one subtype
 - Example: An employee cannot be both full time and part time

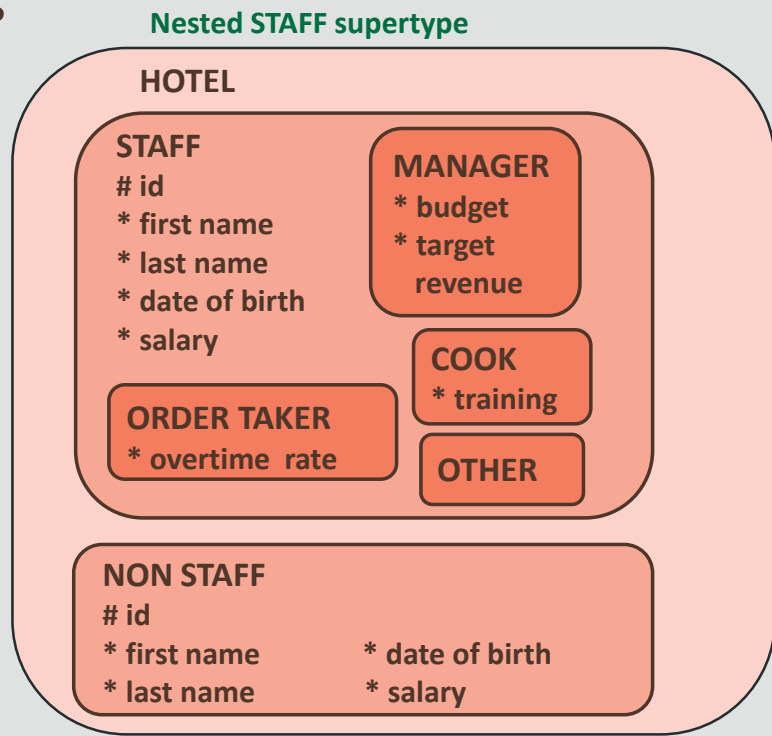
Identifying Subtypes Correctly

- Is this subtype a kind of supertype?
- Have I covered all possible cases? (exhaustive)
- Does each instance fit into one and only one subtype? (mutually exclusive)



Nested Subtypes

- You can nest subtypes



Project Exercise 3

- DFo_3_1_3_Project
 - Oracle Baseball League Store Database
 - Identify and Draw Supertype and Subtype Entities



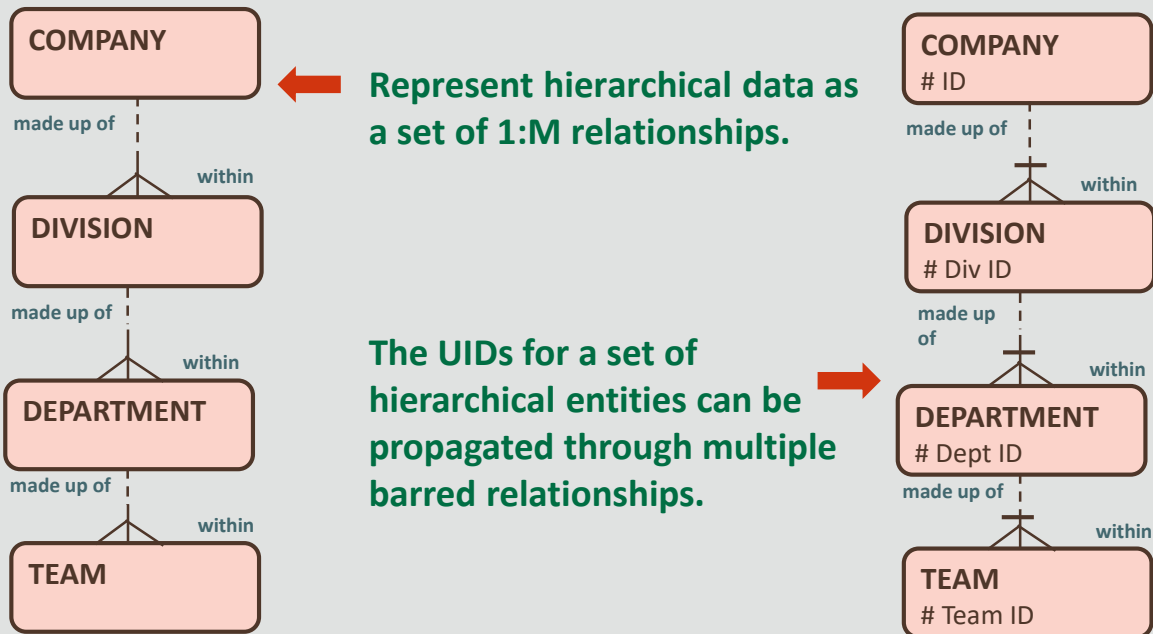
ORACLE
Academy

DFo 3-1
More with Relationships

Copyright © 2020, Oracle and/or its affiliates. All rights reserved.

30

Modeling Hierarchical Data



ORACLE
Academy

DFo 3-1
More with Relationships

Copyright © 2020, Oracle and/or its affiliates. All rights reserved.

31

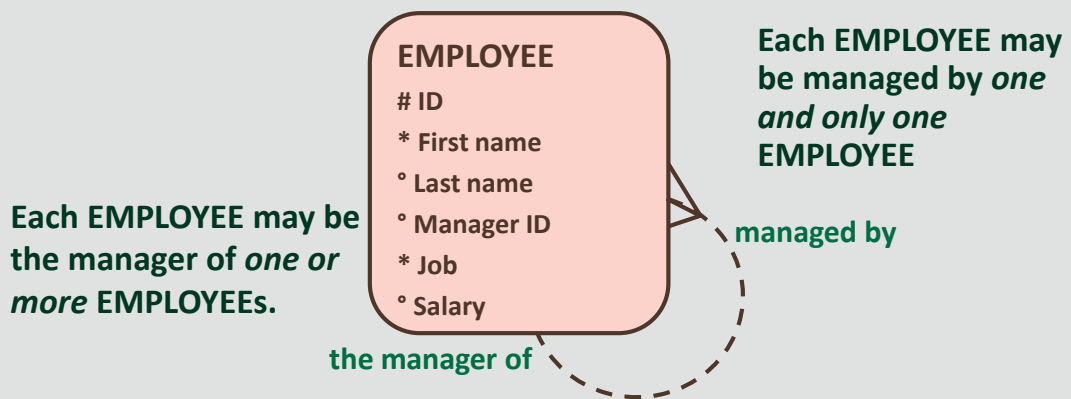
One technique for modeling relationships is to create a set of M:1 relationships hierarchically. In the slide example, an organization has a hierarchical structure:

- A COMPANY may consist of one or more DIVISIONs.
- A DIVISION may consist of one or more DEPARTMENTs.
- A DEPARTMENT may consist of one or more TEAMs.

The UIDs for a set of hierarchical entities may be propagated through multiple relationships. Consider creating artificial attributes to help identify entities in a hierarchical relationship.

Recursive Relationships

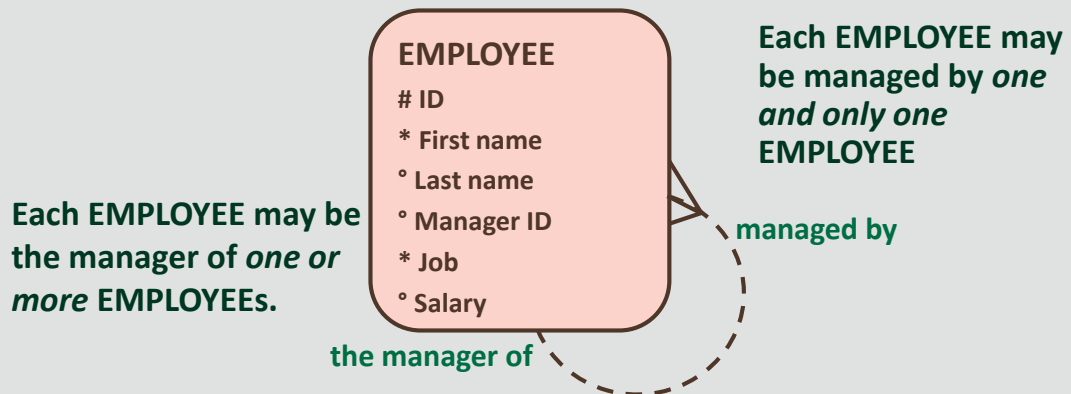
- A recursive relationship is one where an entity instance is related to another instance in the same entity



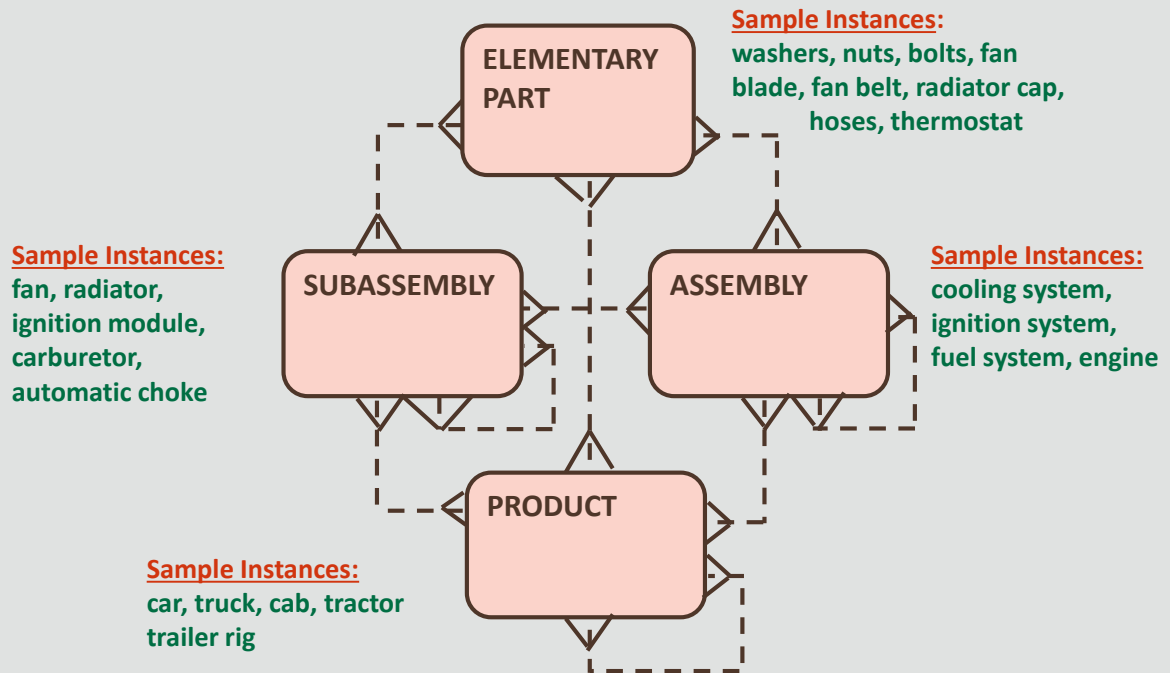
In this example, every manager is also an employee. So manager is not a new entity, but just a subset of the instances of the EMPLOYEE entity. This many-to-one relationship is optional in both directions. It is the same as the relationship between two distinct entities.

Recursive Relationships

- A recursive relationship is always modeled with a loop



Examining Recursive Relationships



ORACLE
Academy

DFo 3-1
More with Relationships

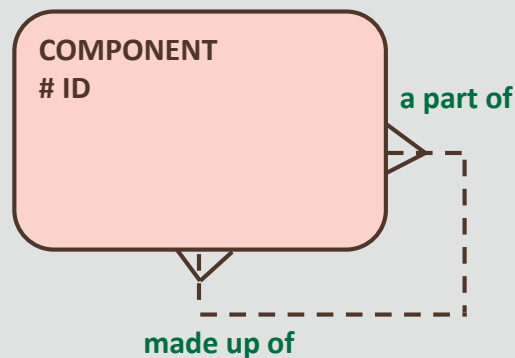
Copyright © 2020, Oracle and/or its affiliates. All rights reserved. 34

An automobile manufacturing organization needs to track elementary parts, subassemblies, assemblies, and products. Bill of Materials data can be modeled with multiple entities for each category of "part" and a set of relationships between each of those entities.

The ERD in the slide depicts the data by considering each of these part categories as an entity.

Generic Modeling

- Another way to model a Bill of Materials recursive relationship is to create a generic COMPONENT entity
- This would create a recursive M:M relationship



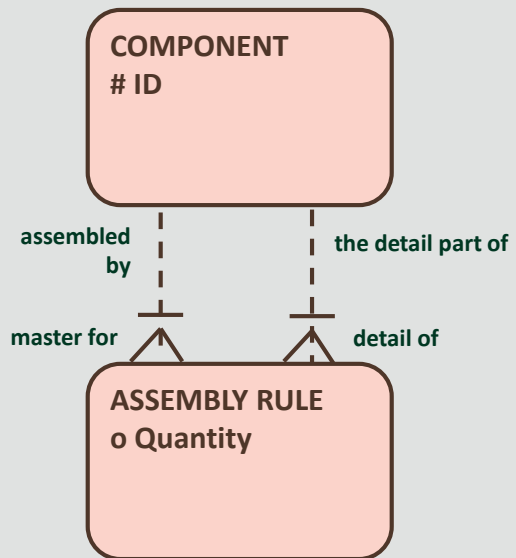
Consider all elementary parts, subassemblies, assemblies, and products as instances of an entity called COMPONENT. Then the complex ERD on the previous page can be remodeled as a simple recursive relationship.

Suppose that some of the instance data supplied in the previous slide is used to validate this model:

- Washers, nuts, and bolts may be part of various SUBASSEMBLIES, ASSEMBLIES, and PRODUCTS.
- A fan blade and a fan belt are part of a fan.
- A radiator cap, host, and thermostat are part of a radiator.
- A cooling system, ignition system, and engine block are part of a complete engine.

Resolving a M:M Recursive Relationship

- Resolve the M:M recursive relationship with an intersection entity



The attribute quantity is associated with the recursive relationship, and therefore must be resolved. Resolve the M:M recursive relationship by adding the ASSEMBLY RULE intersection entity and two 1:M relationships back to the COMPONENT entity. ASSEMBLY RULE will have a Quantity attribute.

Using the instance data again, the ASSEMBLY RULE instance for washers to fan will have a 1:M relationship to the COMPONENT instance for washer and a second 1:M relationship to the COMPONENT instance for fan. The ASSEMBLY RULE entity will record the quantity of washers that are part of a single fan.

Arc Relationship

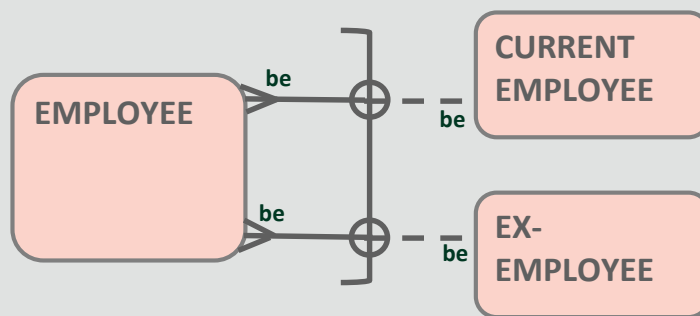
- An arc is an exclusive relationship group, which is defined such that only one of the relationships can exist for any instance of an entity
- The arc drawn between two relationships connects them and demonstrates mutual exclusivity
- The relationship implies an "or" condition
- The arc indicates that any instance of that entity can have only one valid relationship of the relationship in the arc at any one time

Characteristics of an Arc Relationship

- The relationships in an arc frequently have the same relationship name
- The relationships in an arc must be either all mandatory or all optional and should have the same cardinality
- An arc belongs to a single entity and must include only relationships that originate from the entity
- An entity may have multiple arcs, but a specific relationship can participate only in a single arc
- An arc relationship is represented as the arc-shaped line across two or more relationship lines
 - Relationships included in the arc have a circle on the relationship arc line

Arc Relationships

- A supertype entity and its subtypes can be modeled as an arc relationship
- Example: An EMPLOYEE entity is either a CURRENT EMPLOYEE or an EX-EMPLOYEE, but not both (this could also be modeled as a supertype/subtype)



The supertype and subtype (arc) implementation produces one table for every entity. The supertype table has a foreign key for each subtype table. These foreign keys represent exclusive relationships and are optional because only one foreign key can have a value for each row in the table.

Case Scenario: Arc Relationship



Faculty

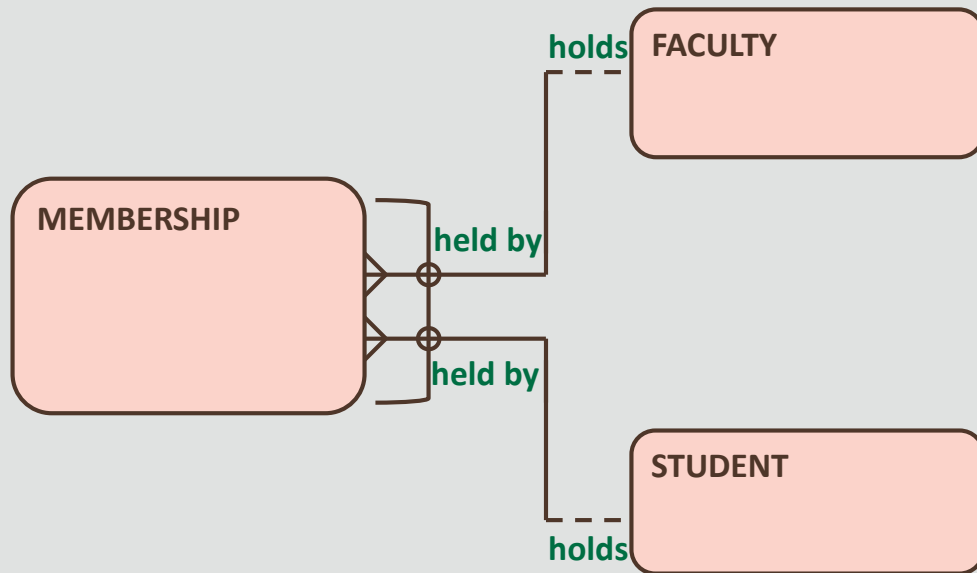
Matt, can you create an entity that can have an arc relationship?

**I can create a common entity called
MEMBERSHIP that would hold
membership details common to all
membership categories**



Matt

Case Scenario: Creating a Common Entity



Each MEMBERSHIP may be held by one and only one FACULTY or it may be held by one and only one STUDENT.

Each FACULTY may be the holder of one or more MEMBERSHIPS. Each STUDENT may be the holder of one or more MEMBERSHIPS. A MEMBERSHIP may not be held by both a FACULTY and a STUDENT. This mutual exclusivity is modeled by the arc.

Arcs may be drawn for optional relationships and for mandatory relationships, but the two relationships participating in the arc must have the same cardinality.

Project Exercise 4

- DFo_3_1_4_Project
 - Oracle Baseball League Store Database
 - Identify Hierarchical, Recursive and Arc Relationships



ORACLE
Academy

DFo 3-1
More with Relationships

Copyright © 2020, Oracle and/or its affiliates. All rights reserved.

42

Summary

- In this lesson, you should have learned how to:
 - Identifying barred relationships
 - Resolve M:M relationships
 - Identify and illustrate nontransferable relationships
 - Identify and draw supertype and subtype entities
 - Identify hierarchical, recursive, and arc relationships



The logo for Oracle Academy is centered on a light gray background. It features the word "ORACLE" in a bold, orange, sans-serif font. Below it, the word "Academy" is written in a smaller, dark gray, sans-serif font. The entire logo is framed by a thin black border, with dark gray horizontal bars at the top and bottom.

ORACLE

Academy

The logo for Oracle Academy is centered on a light gray background. It features the word "ORACLE" in a bold, orange, sans-serif font. Below it, the word "Academy" is written in a smaller, dark gray, sans-serif font. The entire logo is framed by a thin black border, with dark gray horizontal bars at the top and bottom.

ORACLE

Academy

Database Foundations

3-2

Tracking Data Changes

ORACLE
Academy



Copyright © 2020, Oracle and/or its affiliates. All rights reserved.

Objectives

- This lesson covers the following objective:
 - Keep track of data that changes over time



Model Data over Time: Accommodating Historical Data

- Most businesses need to track some historical data to help them find trends and patterns that can be the basis for business innovations or process improvements
- Every update of an attribute or transfer of a relationship means loss of information. Often information is no longer useful, but some systems must keep track of some or all of the historical values of an attribute. In addition, some systems are required to keep an audit trail of each transaction
- Validate any requirements for storing historical data with the user, because storing unnecessary historical data can be costly

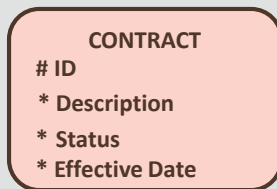
Model Data over Time: Accommodating Historical Data

- Add entities and relationships to the model to accommodate historical data
 - Is an audit trail required?
 - Can attribute values change over time?
 - Can relationships change over time?
 - Do you need to query older data?

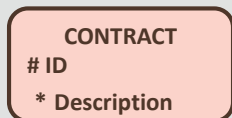


Model Data over Time: Tracking Attribute Values

- Create additional entities to track the attributes' values over time

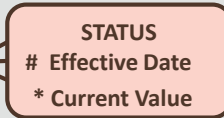


Use one entity if you track only the current status of a contract



contain

state of



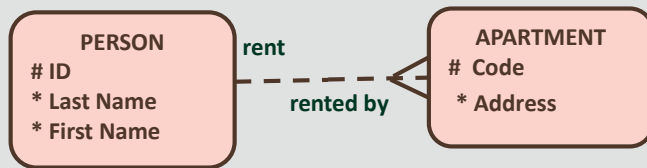
Add an entity to store the status of a contract over time

As shown in the slide example, the **CONTRACT** entity was initially created with four attributes. Because the attributes are contained in one entity, the contract's status and effective date will change to the current status and effective date each time the contract changes. If you are required to track *when* the status of a particular contract changes, create an additional entity and move the status and effective date to the new entity. In this way, the information about when the status of a contract changed can be tracked.

If the status changes more than once on the same day, you should include a current status value attribute in the unique identifier.

Model Data over Time: Accommodating a Changing Relationship

- Add an entity to accommodate a relationship that may change over time

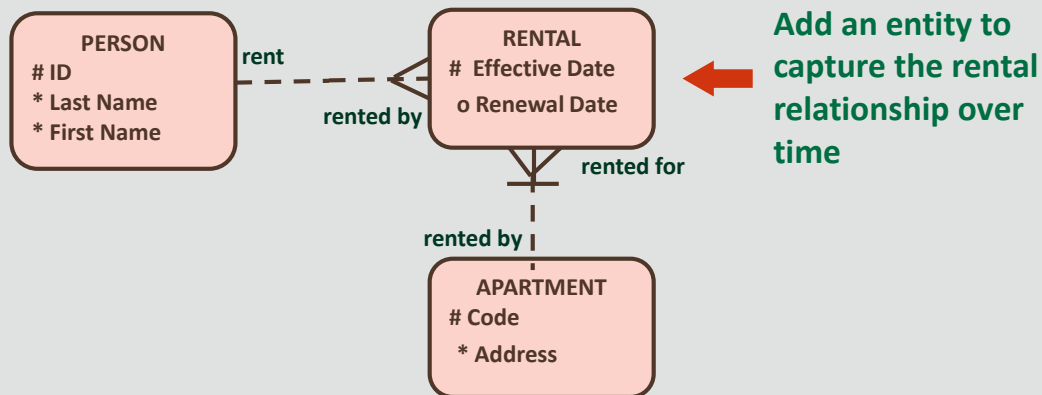


This does not allow tracking of the date that an apartment is rented.

You may need to add an entity to keep track of how a relationship changes over time. In the slide example, a person can rent an apartment, but an apartment can be vacant. If you want to track which apartments are rented at a particular point in time, add an entity to store rental information about an apartment.

Model Data over Time: Accommodating a Changing Relationship

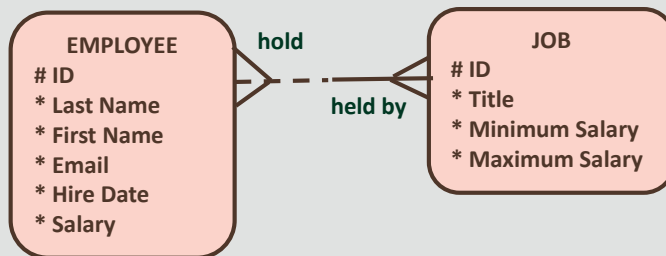
- Add an entity to accommodate a relationship that may change over time



In the RENTAL entity, the apartment code and rental effective date are the unique identifier. You would not include the person ID because you are storing the current and historical rentals. You would not have two rentals for the same apartment with the same effective date.

Model Data over Time: Creating Intersection Entities

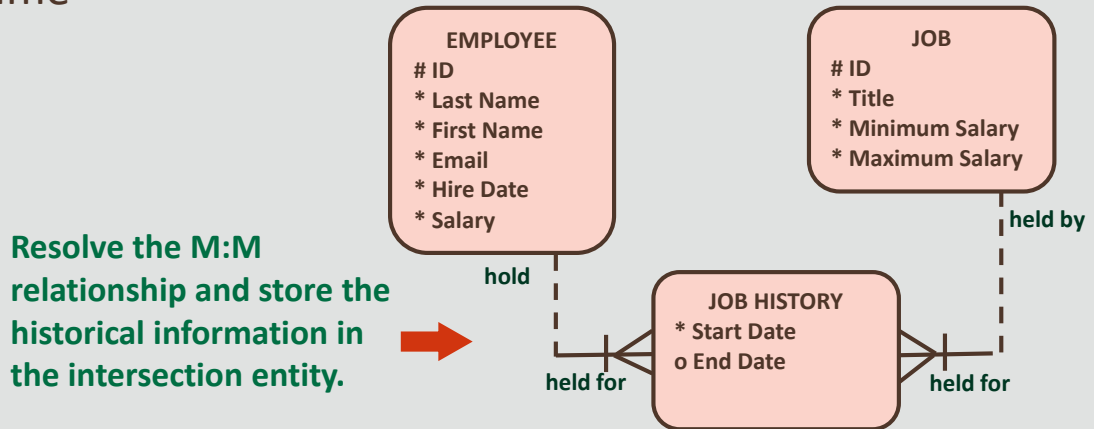
- Resolving M:M relationships with an intersection entity is frequently used to track information about a relationship that changes over time



← This does not store the history of an employee's jobs

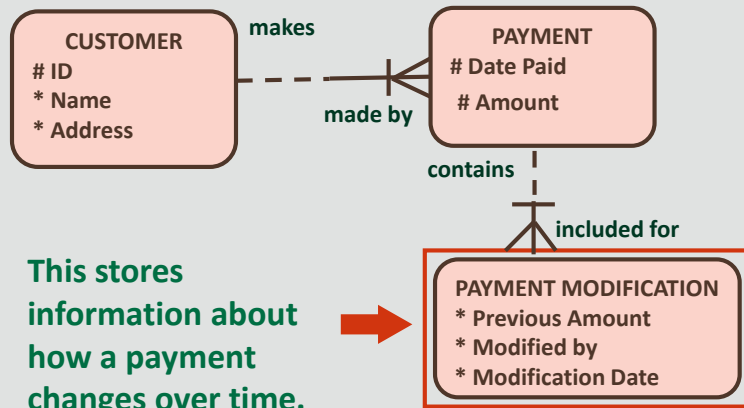
Model Data over Time: Creating Intersection Entities

- An intersection entity is frequently used to track information about a relationship that changes over time



Model Data over Time: Logging Modified Information

- Additional entities and attributes store modified information

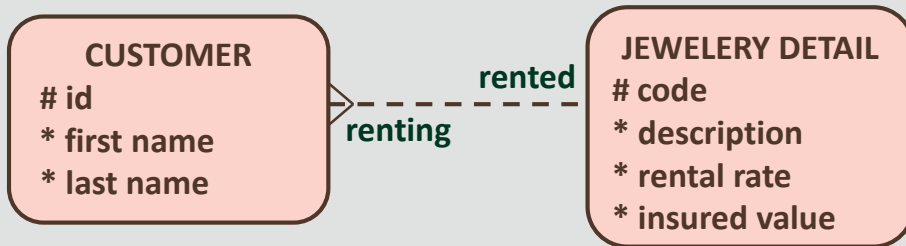


When a system allows a user to modify or remove particular information, you may decide to keep the old values. This is called logging or journaling and is common in financial type models.

In the slide example, a customer may make multiple payments and you can make changes to a payment. If you want to track those changes, you can store the history in the PAYMENT MODIFICATION entity.

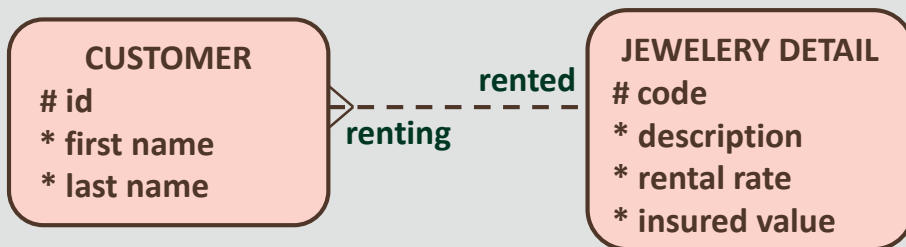
Model Rental over Time: Scenario

- A jewelry store rents necklaces, bracelets, and the like for special occasions, such as award ceremonies and movie premieres
- The store wants to track the rental history



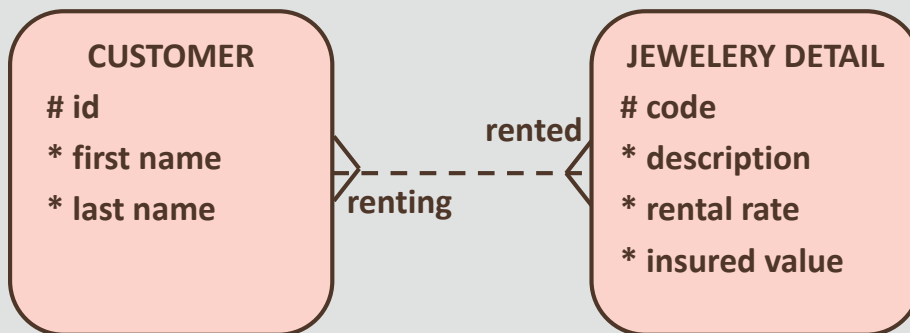
Model Rental over Time: Scenario

- The ERD model tracks only the current renter
- How would you revise the relationship to track the history?



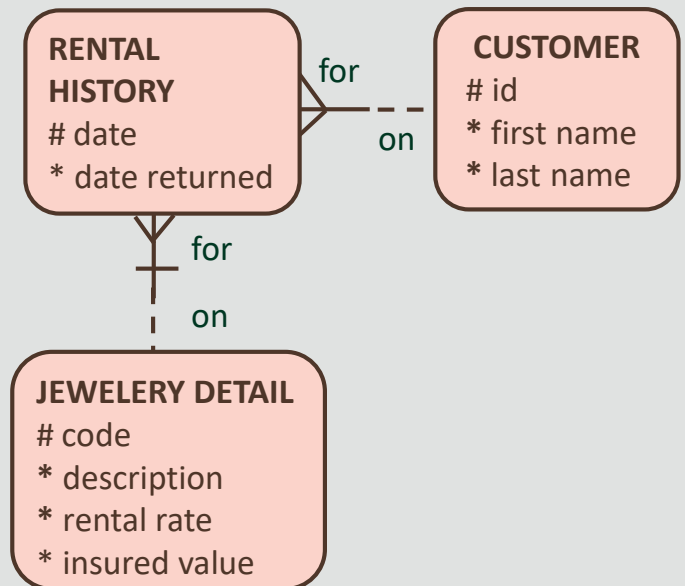
Model Rental over Time: Scenario

- Change the relationship between JEWELRY DETAIL and CUSTOMER to M:M.



Resolve M:M

- Resolve M:M with the RENTAL HISTORY intersection entity
- What is the UID of RENTAL HISTORY?
- What piece of information is not being tracked?



The UID of RENTAL HISTORY is the date rented and the UID of JEWELRY DETAIL (shown by the barred relationship). Customer id is not being tracked in RENTAL HISTORY.

Project Exercise

- DFo_3_2_Project
 - Oracle Baseball League Store Database
 - Keeping Track of Data that Changes over Time



ORACLE
Academy

DFo 3-2
Tracking Data Changes

Copyright © 2020, Oracle and/or its affiliates. All rights reserved.

16

Summary

- In this lesson, you should have learned how to:
 - Keep track of data that changes over time



ORACLE
Academy

DFo 3-2
Tracking Data Changes

Copyright © 2020, Oracle and/or its affiliates. All rights reserved.

17

The logo for Oracle Academy is centered on a light gray background. It features the word "ORACLE" in a bold, orange, sans-serif font. Below it, the word "Academy" is written in a smaller, dark gray, sans-serif font. The entire logo is framed by a thin black border, with dark gray horizontal bars at the top and bottom.

ORACLE

Academy

The logo for Oracle Academy is centered on a light gray background. It features the word "ORACLE" in a bold, orange, sans-serif font. Below it, the word "Academy" is written in a smaller, dark gray, sans-serif font. The entire logo is framed by a thin black border, with dark gray horizontal bars at the top and bottom.

ORACLE

Academy

Database Foundations

3-3

Normalization and Business Rules

ORACLE
Academy



Copyright © 2020, Oracle and/or its affiliates. All rights reserved.

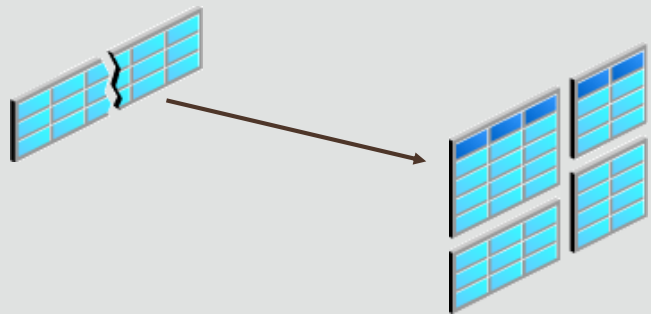
Objectives

- This lesson covers the following objectives:
 - Explain normalization
 - Describe the normal forms
 - Use normalization to validate data
 - Describe business rules



Normalization

- Is the process of organizing the attributes and tables of a relational database to minimize redundancy
- Helps in handling insert, update, and delete anomalies, ensuring a better performance of the database



Normalization can be defined as the process of decomposing a table with inconsistencies to produce a smaller, well-structured table. Normalization may involve splitting a table into two or more tables and defining relationships between the tables. The objective is to isolate data so that additions, deletions, and modifications of an attribute or a field can be made in just one table and then propagated throughout the rest of the database by using well-defined relationships. Edgar F. Codd, the inventor of the relational model, introduced the concept of normalization.

Why Should You Normalize Data?

- Reduce redundant data in the existing design
- Increase the integrity of data and the design's stability
- Eliminate other types of data inconsistencies and anomalies
- Identify missing tables, columns, and constraints



No Duplicate Content!



ORACLE
Academy

DFO 3-3
Normalization and Business Rules

Copyright © 2020, Oracle and/or its affiliates. All rights reserved.

5

What Is Normalization?

- Normalization is a relational database concept, but its principles apply to data modeling
- The goal is to normalize data to 3NF before transforming the model to your relational design

Rule	Description
First Normal Form (1NF)	All attributes must be single-valued.
Second Normal Form (2NF)	An attribute must be dependent on its entity's entire UID.
Third Normal Form (3NF)	No non-UID attributes can be dependent on another non-UID attribute.

Project Exercise 1

- DFo_3_3_1_Project
 - Oracle Baseball League Store Database
 - Converting the Data into Unnormalized Form



Types of Normal Forms

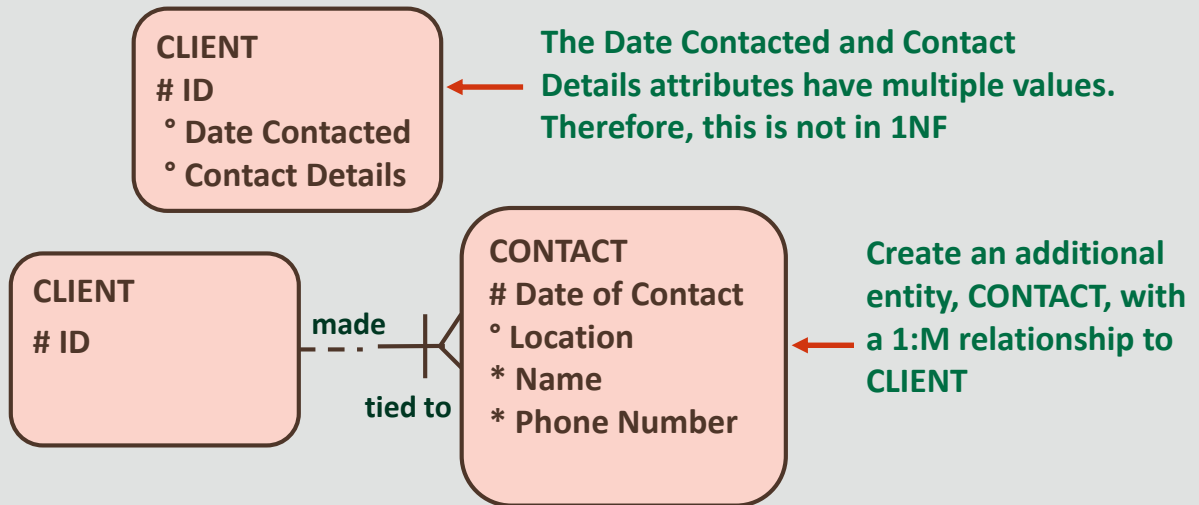
- Main types of normal forms introduced by E.F. Codd:
 - First Normal Form (1NF)
 - Second Normal Form (2NF)
 - Third Normal Form (3NF)

What is First Normal Form? (1NF)

- First Normal Form requires that no multi-valued attributes exist
- To check for 1NF, validate that each attribute has a single value for each instance of the entity
- If an attribute is multi-valued, create an additional entity and relate it to the original entity with a 1:M relationship

First Normal Form (1NF)

- Each attribute must have a single value for each occurrence of the entity



Project Exercise 2

- DFo_3_3_2_Project
 - Oracle Baseball League Store Database
 - First Normal Form

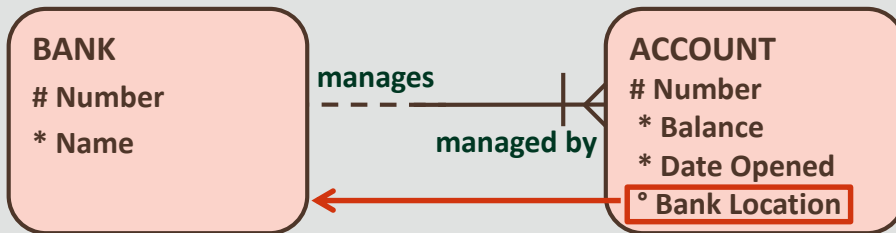


What is Second Normal Form? (2NF)

- Second Normal Form (2NF) requires that any non-UID attribute be dependent on (be a property of, or a characteristic of) the entire UID
- If the UID is a composite UID each attribute must be dependent on all parts of the composite UID
- If an attribute is not dependent on the entire UID, create an additional entity with the partial UID

Second Normal Form (2NF)

- An attribute must be dependent on its entity's entire UID



The Bank Location attribute is dependent on BANK , not on ACCOUNT. Therefore, this is not in 2NF. Move the attribute to the BANK entity.

Project Exercise 3

- DFo_3_3_3_Project
 - Oracle Baseball League Store Database
 - Second Normal Form

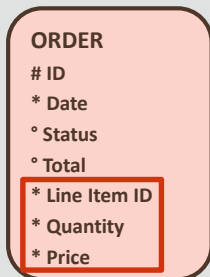


What is Third Normal Form? (3NF)

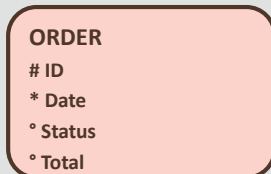
- The rule of Third Normal Form (3NF) states that no non-UID attribute can be dependent on another non-UID attribute
- Third Normal Form prohibits transitive dependencies
- A transitive dependency exists when any attribute in an entity is dependent on any other non-UID attribute in that entity
- You need to move any non-UID attribute that is dependent on another non-UID attribute into a new entity

Third Normal Form (3NF)

- Each attribute depends only on the UID of its entity
- Move any non-UID attribute that is dependent on another non-UID attribute into a new entity

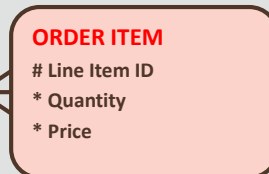


The Quantity and Price attributes are dependent on the ORDER table ID (UID) and Line Item ID (non-UID). Therefore, this is not in 3NF.



contains

included in



Create a new ORDER ITEM entity. Move the Line Item ID, Quantity, and Price attributes to the new entity, and then create an identifying relationship.

Project Exercise 4

- DFo_3_3_4_Project
 - Oracle Baseball League Store Database
 - Third Normal Form



Business Rules

- A business rule is a statement that defines or constrains some aspect of the business
- It is used to define entities, attributes, relationships, and constraints
- It includes rules and policies that define how a business operates and handles its day-to-operations

Business Rules

- There are two types of business rules:
 - Structural
 - Procedural
- Example: All orders must have a Contact Number

The example in the slide is a structural business rule.

Sources of Business Rules

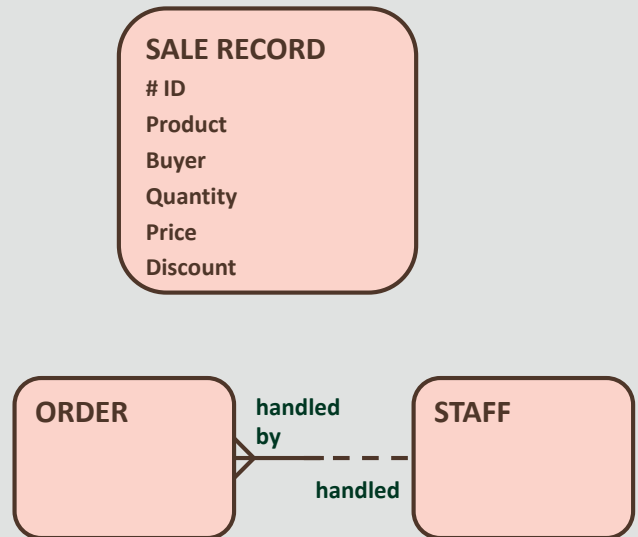
- Top management and managers
- Written documentation
 - Procedures
 - Standards
 - Operations manuals
- Direct interviews with end users

Structural Business Rules

- Structural business rules indicate the types of information to be stored and how the information elements interrelate
- Structural business rules detail a specific, static aspect of the business
- These rules can always be diagrammed in the ERD

Structural Business Rules: Examples

- A sales record includes product, buyer, quantity, price, and discount
- All restaurant orders must be handled by a staff member (specifically, an order taker)



Library Case Scenario – Structural Rules

Is it
reasonable/effective
for two members to
have the same
member ID or no
member ID?

Is it
reasonable/effective
for two books to have
the same book ID
number or no book ID
number at all?



Procedural Business Rules

- Procedural rules deal with the prerequisites, steps, processes, or workflow requirements of a business
- Many procedural business rules are related to time: Event A must happen before event B
- Some procedural business rules cannot be diagrammed, but must still be documented so that they can be programmed later

Procedural Business Rules: Examples

- An online store might not accept a next-day delivery order if the order is received after 3:00 PM
- Approval for travel requests to a conference must be signed by the manager

Library Case Scenario – Procedural Rules

**Books must be
returned to the
library within
30 days**

**Members who do
not return the book
by the due date will
not be permitted to
borrow another
book**



Additional Programming

- Not all business rules can be modeled when a conceptual data model is being developed. Some rules, such as these must be implemented by programming as processes that interact with data:
 - Any employee whose overtime exceeds 10 hours per week must be paid 1.5 times the hourly rate
 - OR
 - Customers whose account balances are 90 days overdue will not be permitted to charge additional orders

Summary

- In this lesson, you should have learned how to:
 - Explain normalization
 - Describe the normal forms
 - Use normalization to validate data
 - Describe business rules



The logo for Oracle Academy is centered on a light gray background. It features the word "ORACLE" in a bold, orange, sans-serif font. Below it, the word "Academy" is written in a smaller, dark gray, sans-serif font. The entire logo is framed by a thin black border, with dark gray horizontal bars at the top and bottom.

ORACLE

Academy

The logo for Oracle Academy is centered on a light gray background. It features the word "ORACLE" in a bold, orange, sans-serif font. Below it, the word "Academy" is written in a smaller, dark gray, sans-serif font. The entire logo is framed by two horizontal dark gray bars, one at the top and one at the bottom.

ORACLE

Academy

Database Foundations

3-4

Data Modeling Terminology and Mapping

ORACLE
Academy



Copyright © 2020, Oracle and/or its affiliates. All rights reserved.

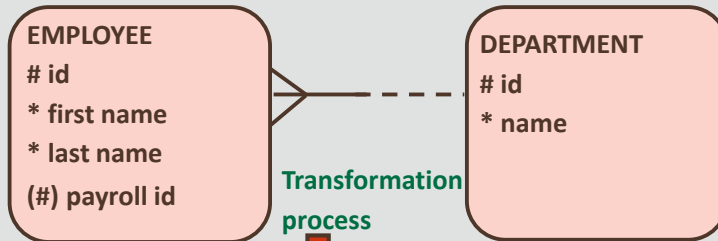
Objectives

- This lesson covers the following objectives:
 - Apply terminology mapping between the logical and physical models
 - Understand and apply the Oracle naming conventions for tables and columns used in physical models
 - Apply the rules of relationship mapping to correctly transform relationships



Logical To Physical Transformation: Example

Logical Model
(ERD)



Transformation
process

EMPLOYEES (EPE)

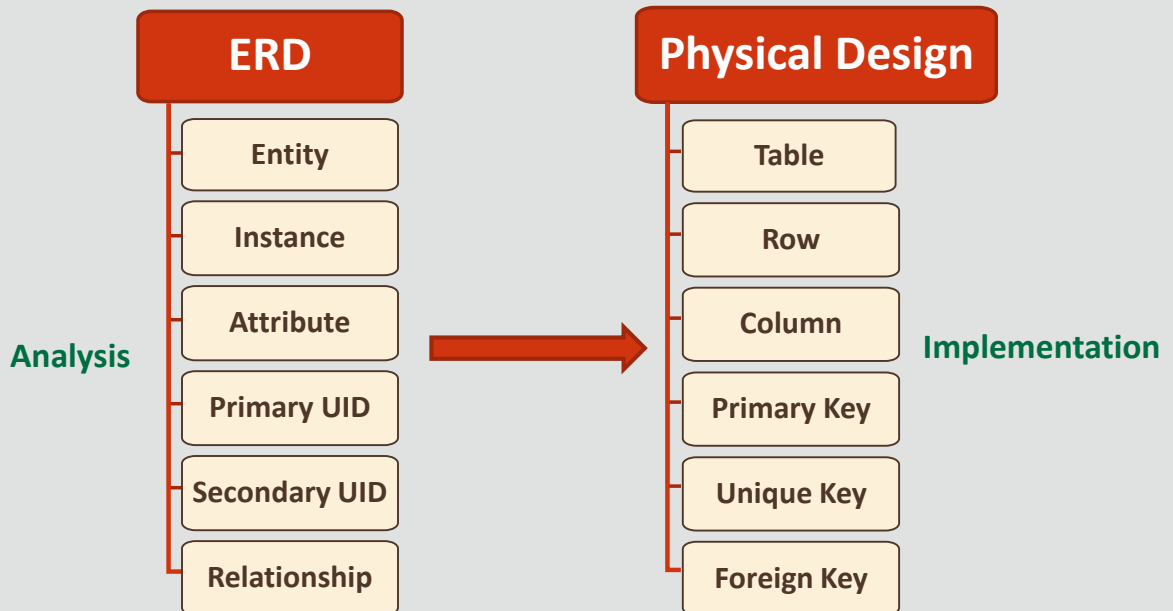
Key Type	Optionality	Column name
pk	*	id
uk	*	payroll_id
	*	last_name
	*	first_name
fk	*	department_id

Physical Implementation:
Relational Database

DEPARTMENTS (DPT)

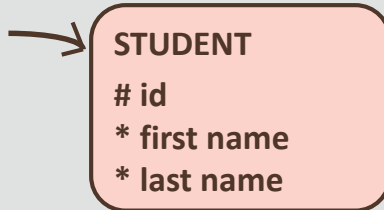
Key Type	Optionality	Column name
pk	*	id
	*	name

Terminology Mapping



An Entity and a Corresponding Table

Entity



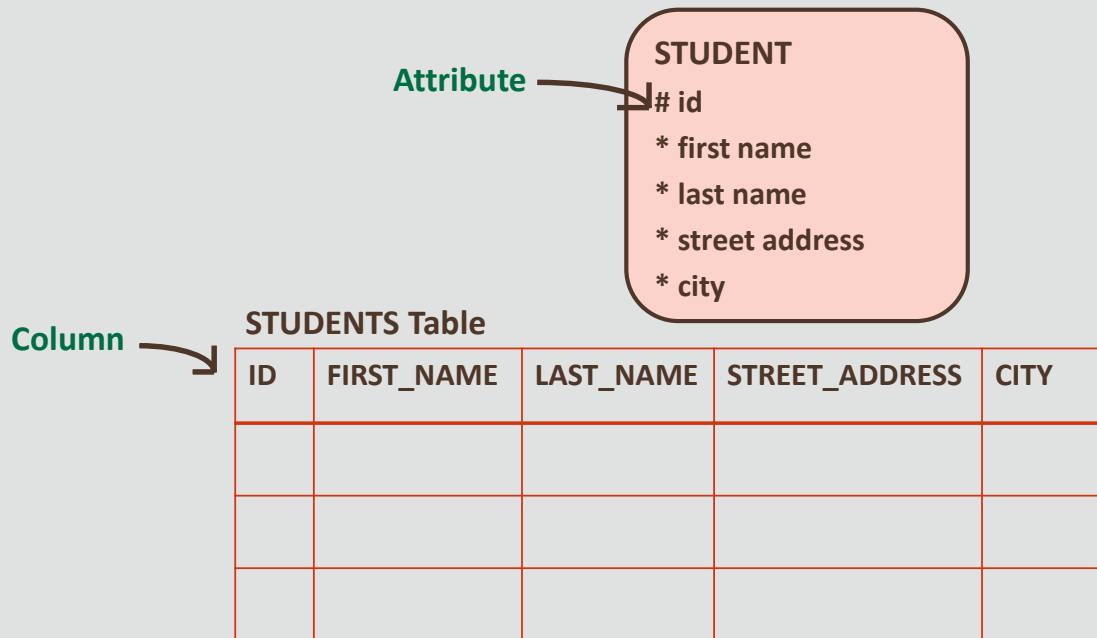
Table

STUDENTS Table

A rectangular table with three columns and four rows. The columns are labeled **ID**, **FIRST_NAME**, and **LAST_NAME**. The first row contains the column headers, and the following three rows are empty. A green arrow points from the word "Table" to the top-right corner of the table.

ID	FIRST_NAME	LAST_NAME

Attributes and Corresponding Columns



ORACLE
Academy

DFo 3-4
Data Modeling Terminology and Mapping

Copyright © 2020, Oracle and/or its affiliates. All rights reserved.

7

Attributes are classified as one of the following:

- **Not null (mandatory):** Indicated by the asterisk (*) symbol next to the attribute
- **Optional (nulls allowed):** Indicated by the o (optional) symbol next to the attribute

An Instance and a Corresponding Row

Entity
STUDENT

Instance
J Smith

ID	FIRST_NAME	LAST_NAME	STREET_ADDRESS	CITY
101	Sam	Linkin	99B, Chuah Street	LA
102	Neena	Markin	44A, Church Street	NZ
103	Rick	Austina	1 st Cross, Palm Street	SA
104	J	Smith	Alpha Street	CA

→ **Row**

Table Diagram Notations

- A table diagram is additional documentation that is often used to further explain keys and columns in the physical database

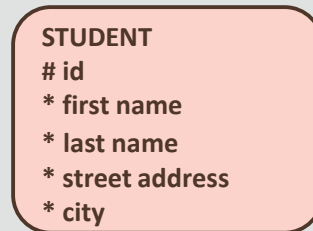
STUDENTS Table

Key Type	Optionality	Column Name
pk	*	id
	*	first_name
	*	last_name
	*	street_address
	*	city

- The Key Type column should contain values of "pk" for the primary key, "uk" for the unique key, or "fk" for the foreign key column. The cell is blank if the column is not a part of a key.
- The Optionality column must contain an asterisk (*) if the column is mandatory and a lowercase "o" if it is optional. This is similar to the ERD.
- The third column is for the column name.

Naming Conventions for Tables

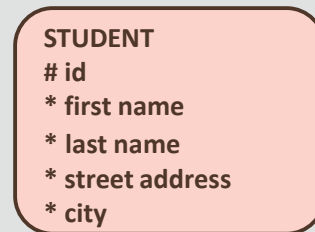
- The table name is the plural of the entity name.
 - Example: STUDENT becomes STUDENTS.
- Column names are identical to the attribute names, except that special characters and spaces are replaced with underscores



STUDENTS		
Key Type	Optionality	Column name
pk	*	id
	*	first_name
	*	last_name
	*	street_address
	*	city

Naming Conventions for Columns

- Column names are similar to the attribute names except that special characters and spaces are replaced with underscores
- Column names often use more abbreviations than attribute names.
 - Example: First name becomes first_name or fname



STUDENTS		
Key Type	Optionality	Column name
pk	*	id
	*	first_name
	*	last_name
	*	street_address
	*	city

Table Short Names

- A unique short name for every table is useful when naming foreign key columns

PRIVATE HOME

id

* address

o comments

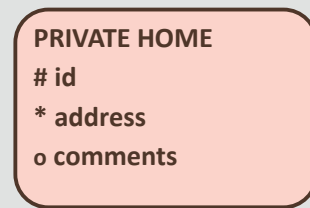


PRIVATE_HOMES (PHE)

Key Type	Optionality	Column name
pk	*	id
	*	address
	o	comments

Table Short Names

- Create short names based on:
 - Entity names that contain more than one word
 - Entity names that contain one word but more than one syllable
 - Entity names that contain one syllable but more than one character



PRIVATE_HOMES (PHE)		
Key Type	Optionality	Column name
pk	*	id
	*	address
	o	comments

For entity names that contain more than one word, take the:

- First character of the first word
- First character of the second word
- Last character of the last word

Example: PRIVATE HOME gets a short name of PHE.

For entity names that contain one word but more than one syllable, take the:

- First character of the first syllable
- First character of the second syllable
- Last character of the last syllable

Example: EMPLOYEE gets a short name of EPE and CLIENT gets a short name of CET.

For entity names that contain one syllable but more than one character, take the:

- First character
- Second character
- Last character

Example: FLIGHT gets a short name of FLT.

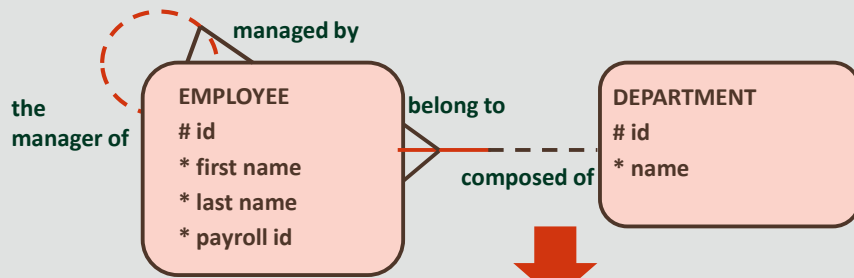
Naming Restrictions with Oracle

- Table and column names:
 - Must start with a letter
 - Can contain up to 30 alphanumeric characters
 - Cannot contain spaces or special characters such as "!", " but "\$," "#," and "_" are permitted
 - Cannot be “reserved words” in Oracle DB or SQL
- Table names must be unique within one user account in the Oracle database
- Column names must be unique within a table

Relationship Mapping

- Relationships are mapped between primary keys and foreign keys to allow one table to reference another
- A relationship creates one or more foreign-key columns in the table on the many side of the relationship
- We use the short name of the table to name the foreign-key column
- In the example on the next page, the foreign-key column in the EMPLOYEES table is dpt_id for the relationship with DEPARTMENT, and mgr_id for the recursive relationship with itself

Relationship Mapping Illustrated



EMPLOYEES (EPE)

Key Type	Optionality	Column name
pk	*	id
	*	first_name
	*	last_name
uk	*	payroll_id
fk1	*	dpt_id
fk2	o	mgr_id

DEPARTMENTS (DPT)

Key Type	Optionality	Column name
pk	*	id
uk	*	name

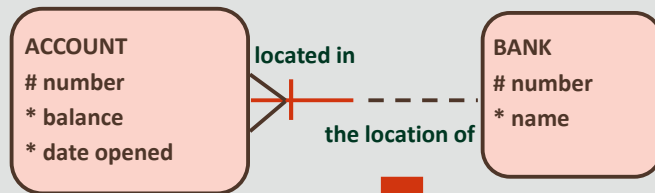
foreign key refers to

foreign key refers to

Mapping of Barred Relationships

- A barred relationship is mapped to a foreign-key column on the many side, just like any other 1:M relationship
- In this case, the foreign-key column plays a double role because it is also part of the primary key
- In the example, bak_number is a foreign-key column in ACCOUNTS that refers to the primary key of BANKS
- It is also part of the primary key of ACCOUNTS

Mapping of Barred Relationships



ACCOUNTS (ACT)

Key Type	Optionality	Column name
pk	*	number
	*	balance
	*	date_opened
pk,fk	*	bak_nbr

BANKS (BAK)

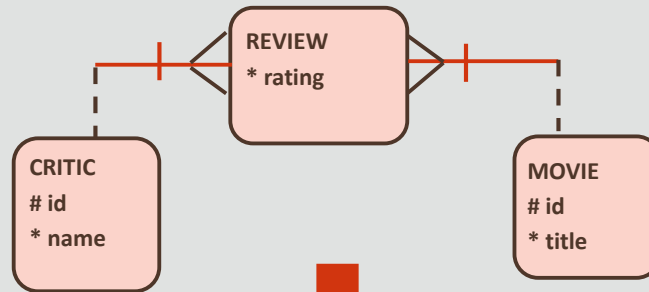
Key Type	Optionality	Column name
pk	*	number
	*	name

foreign key refers to

Mapping Many-to-Many Relationships

- A M:M relationship is resolved with an intersection entity, which maps to an intersection table
- This intersection table will contain foreign-key columns that refer to the originating tables
- In the example, REVIEWS contains all the combinations that exist between a CRITIC and a MOVIE

Mapping Many-to-Many Relationships



REVIEWS (RVW)

Key Type	Optionality	Column name
pk,fk1	*	ctc_id
pk,fk2	*	mve_id
	*	rating

CRITICS (CTC)

Key Type	Optionality	Column name
pk	*	id
	*	name

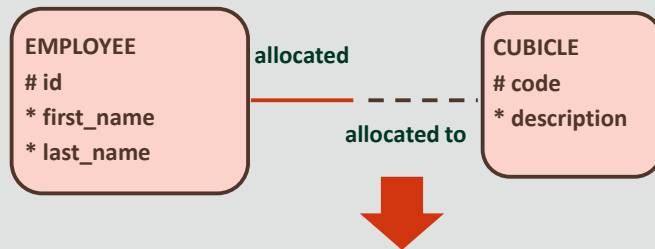
MOVIES (MVE)

Key Type	Optionality	Column name
pk	*	id
	*	title

Mapping One-to-One Relationships

- When transforming a 1:1 relationship, you create a foreign key and a unique key
- All columns of this foreign key are also part of the unique key
- If the relationship is mandatory on one side, the foreign key is created in the corresponding table
- In the example, cbe_code is the foreign-key column in EMPLOYEES that refers to the primary key of CUBICLES.
- Cbe_code would also be unique within the EMPLOYEES table

Mapping One-to-One Relationships



EMPLOYEES (EPE)

Key Type	Optionality	Column name
pk	*	id
	*	first_name
	*	last_name
fk,uk	*	cbe_code

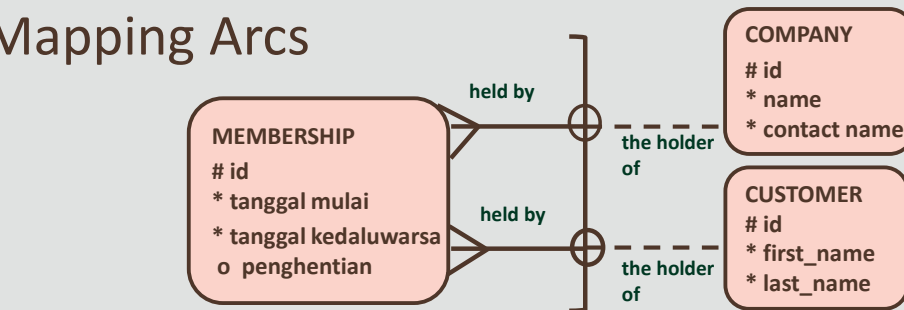
CUBICLES (CBE)

Key Type	Optionality	Column name
pk	*	code
	*	description

Mapping Arcs

- The entity that has the arc will map to a table that contains foreign keys from the tables on the “one” end of the relationships
- Note that even if the relationships in the arc are mandatory on the many side, the resulting foreign keys have to be optional (because one of them will always be blank). A check constraint stored in the database can easily do this

Mapping Arcs



MEMBERSHIPS (MBP)

Key Type	Optionality	Column name
pk	*	id
	*	start_date
	*	expiration_date
	o	termination
fk1	o	cpe_id
fk2	o	cms_id

COMPANIES (CPE)

Key Type	Optionality	Column name
pk	*	id
	*	name
	*	contact_name

CUSTOMERS (CMS)

Key Type	Optionality	Column name
pk	*	id
	*	first_name
	*	last_name

ORACLE
Academy

DFo 3-4
Data Modeling Terminology and Mapping

Copyright © 2020, Oracle and/or its affiliates. All rights reserved.

24

Since the arc represents exclusive relationships, additional code is needed to enforce that only one of the foreign keys has a value for every row in the table.

A check constraint stored in the database can easily do this.

In the example, the code for the check constraint would look like this:

- CHECK (cpe_id is not null AND cms_id is null)
- OR (cpe_id is null AND cms_id is not null)

If the relationships were fully optional, you would add: OR (cpe_id is null AND cms_id is null)

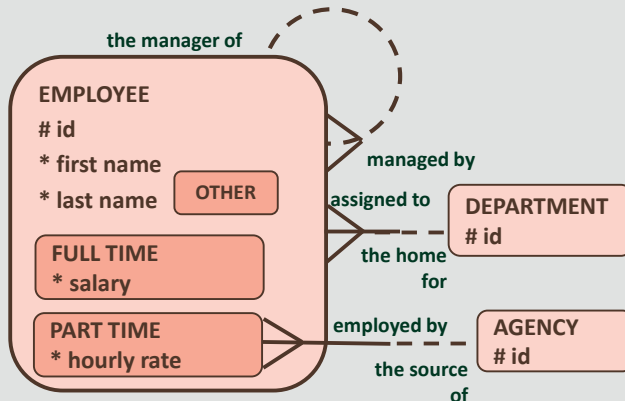
Mapping Supertype/Subtypes

- Supertype/subtype entities can be mapped in multiple ways :
 - Single table implementation : one table is created regardless of number of subtypes, used when most of the attributes and relationships are shared and therefore at the supertype level
 - Two table implementation : a table is created for each of the subtypes (so there can be more than two tables), used when subtypes have little in common and few shared attributes and relationships

Single Table Implementation

- The single table gets one column for each attribute of the supertype, along with the original optionality of the attribute
- The table also gets a column for each attribute belonging to the subtype, but the columns all become optional
- Additionally, a mandatory column should be created to act as a discriminator column to distinguish between the different subtypes of the entity

Single Table Implementation



DEPARTMENTS (DPT)

Key Type	Optionality	Column name
pk	*	id

AGENCIES (AGY)

Key Type	Optionality	Column name
pk	*	id

EMPLOYEES (EPE)

Key Type	Optionality	Column name
pk	*	id
	*	first_name
	*	last_name
	o	salary
	o	hourly_rate
fk1	*	dpt_id
fk2	o	agy_id
	*	epe_type
fk3	o	mgr_id

ORACLE
Academy

DFo 3-4
Data Modeling Terminology and Mapping

Copyright © 2020, Oracle and/or its affiliates. All rights reserved. 27

Identifiers: Unique identifiers transform into primary and unique keys.

Relationships: Relationships at the supertype level transform as usual. Relationships at the subtype level are implemented as optional foreign-key columns.

Integrity constraints: A check constraint is needed to ensure that for each particular subtype, all columns that come from mandatory attributes are not null.

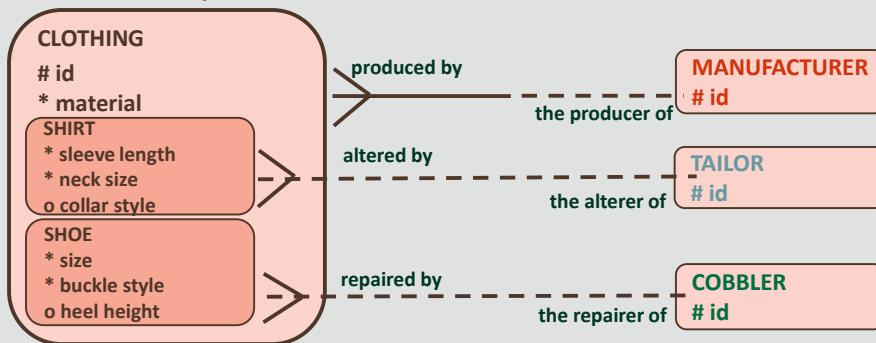
In the logical model, salary is mandatory for full-time employees and hourly rate is mandatory for part-time employees.

- When the EMPLOYEE supertype is implemented as a single table in the physical model, these attributes become optional.
- A check constraint is needed to enforce the business rules modeled in the ERD.

Two Table Implementation

- One table per first-level subtype
- Each table gets one column for each attribute of the supertype along with its original optionality
- Each table also gets one column for each attribute belonging to the subtype along with its original optionality

Two Table Implementation



SHIRTS (SHT)

Key Type	Optionality	Column name
pk	*	id
	*	material
	*	sleeve_length
	*	neck_size
	o	collar_style
fk1	o	tlr_id
fk2	*	mnr_id

SHOES (SHE)

Key Type	Optionality	Column name
pk	*	id
	*	material
	*	size
	*	buckle_style
	o	heel_height
fk1	o	clr_id
fk2	*	mnr_id

refers to **tailors**

refers to **manufacturers**

refers to **cobblers**

ORACLE
Academy

DFo 3-4
Data Modeling Terminology and Mapping

Copyright © 2020, Oracle and/or its affiliates. All rights reserved. 29

Identifiers: The primary UID at the supertype level creates a primary key for each table. Secondary UIDs of the supertype become unique keys in each table.

Relationships:

- All tables get a foreign key for a relationship at the supertype level, with the original optionality (manufacturers)
- For relationships at the subtype levels, the foreign key is implemented in the table it is mapped to, original optionality is retained. (tailors, cobblers)

Project Exercise

- DFo_3_4_Project
 - Oracle Baseball League Store Database
 - Apply the Rules of Relationship Mapping to Transform Relationships



Summary

- In this lesson, you should have learned how to:
 - Apply terminology mapping between the logical and physical models
 - Understand and apply the Oracle naming conventions for tables and columns used in physical models
 - Apply the rules of relationship mapping to correctly transform relationships



The logo for Oracle Academy is centered on a light gray background. It features the word "ORACLE" in a bold, orange, sans-serif font. Below it, the word "Academy" is written in a smaller, dark gray, sans-serif font. The entire logo is framed by a thin black border, with dark gray horizontal bars at the top and bottom.

ORACLE

Academy